

RAG CHATBOT PROJECT

COMPLETE INTERVIEW PREPARATION SHEET v2

Fresher Level • 59 Questions • WHY questions + Model Answers • Score Sheet
Python • LangChain • LlamaIndex • FAISS • FastAPI • Streamlit • Docker

Blue Q — concept / implementation

Orange WHY Q — design decision / trade-off

Green box = ideal answer for every question

Section 1 — Project Overview

Big picture understanding — communication + architecture

Q1

Walk me through this project in 60 seconds as if explaining to a non-technical person.

[Communication]

IDEAL ANSWER

I built a chatbot that lets you upload any document — PDF, Word file, CSV, or a webpage link — and ask questions about it. Unlike ChatGPT, it only answers from your actual document and shows you which page the answer came from.

Score: ____ /5

Red flag:

Notes: _____

Q2

What problem does RAG solve that plain ChatGPT cannot?

[Concept]

IDEAL ANSWER

Plain GPT has a knowledge cutoff and no access to private data. RAG retrieves relevant chunks from your own documents before generating, grounding every answer in real content and eliminating hallucinations about your data.

Score: ____ /5

Red flag:

Notes: _____

Q3

WHY?

Why build a RAG chatbot instead of a simpler project?

vs: simple chatbot vs RAG

IDEAL ANSWER

RAG combines file handling, vector databases, API design, LLMs, memory, and frontend deployment — more skills demonstrated in one project than a basic CRUD app. It also solves a real business problem companies pay for.

Score: ____ /5

Notes: _____

Q4

What is the complete data flow from file upload to answer?

[Architecture]

IDEAL ANSWER

Upload → format detection (PDF/DOCX/CSV/URL) → format-aware chunking → OpenAI embeddings → FAISS index merge → user question → embed question → FAISS top-k retrieval → chunks + chat history stuffed into prompt → GPT generates answer → answer + source citations returned to UI.

Score: ____ /5

Red flag:

Notes: _____

Q5**Which day did you find hardest and how did you debug it?***[Debugging]***IDEAL
ANSWER**

Day 10 — FAISS merge_from() silently dropped metadata when indexes had different field structures. I debugged by printing chunk metadata after merge and found the enrichment pipeline wasn't running on one loader. Fixed by routing all loaders through the same metadata.py step.

Score:
___/5**Red flag:**

Notes: _____

Section 2 — Embeddings & Vector Search

*Day 2–4 — expect the deepest questions here***Q6****What is an embedding and what does text-embedding-3-small output?***[Concept]***IDEAL
ANSWER**

An embedding converts text into a list of ~1536 floating-point numbers representing semantic meaning. Similar sentences produce vectors that point in similar directions in this high-dimensional space, enabling mathematical similarity comparison.

Score:
___/5**Red flag:**

Notes: _____

Q7**WHY?****Why FAISS instead of ChromaDB?***vs: FAISS vs ChromaDB vs Pinecone***IDEAL
ANSWER**

FAISS is a pure in-memory C++ library — zero server setup, extremely fast, just two files on disk. ChromaDB needs a running server and adds metadata filtering but that complexity isn't needed for a single-user demo. I'd switch to ChromaDB or Pinecone for multi-user apps needing filtering.

Score:
___/5

Notes: _____

Q8**WHY?****Why text-embedding-3-small instead of ada-002 or 3-large?***vs: 3-small vs ada-002 vs 3-large***IDEAL
ANSWER**

text-embedding-3-small outperforms ada-002 on most benchmarks at lower cost — it's the best accuracy-to-price ratio. 3-large is ~13% more accurate but costs 5x more per token. For a portfolio demo, small is the smart default.

Score:
___/5

Notes: _____

Q9**What is cosine similarity and why use it for text?***[Concept]***IDEAL
ANSWER**

Cosine similarity measures the angle between two vectors — not size but direction. A long sentence and a short sentence saying the same thing have different magnitudes but point the same direction, so cosine gives them a high similarity score. Perfect for semantic text matching.

Score:
___/5**Red flag:**

Notes: _____

Q10

WHY?

Why k=4 in similarity_search instead of k=1 or k=20?

vs: k=4 vs k=1 vs k=20

**IDEAL
ANSWER**

k=1 misses context spread across multiple paragraphs. k=20 floods the prompt with irrelevant content — the LLM gets confused (lost-in-the-middle problem) and token cost spikes. k=4 with 500-char chunks fits ~2000 chars of context, enough for most answers without noise.

Score: ____ /5

Notes: _____

Q11

WHY?

Why persist the FAISS index to disk instead of rebuilding on every startup?

vs: rebuild every time vs persist

**IDEAL
ANSWER**

Each embedding call costs money. A 100-page PDF produces ~400 chunks = 400 API calls every restart. Saving index.faiss and index.pkl means embed once, reload instantly forever. The two files together reconstruct the full index including text and metadata.

Score: ____ /5

Notes: _____

Q12**If the user asks something not in the document, what does FAISS return?***[Edge Cases]***IDEAL
ANSWER**

FAISS always returns k results regardless of relevance — it finds the least-dissimilar chunks. The system prompt handles this: 'If the answer is not in the context, say I don't have that information.' This prevents the LLM from hallucinating an answer from its training data.

Score: ____ /5

Red flag: Notes: _____

Section 3 — Document Ingestion & Chunking

*Day 3, 8–9 — format-aware splitting pipeline***Q13**

WHY?

Why chunk documents instead of embedding the whole file?

vs: whole file vs chunks

**IDEAL
ANSWER**

LLMs have context window limits (GPT-3.5 = 16k tokens). A 200-page PDF doesn't fit in one prompt. Also, one embedding for a whole document loses precision — a question about page 50 is drowned by signal from pages 1–200. Smaller chunks = sharper, more accurate retrieval.

Score: ____ /5

Notes: _____

Q14

WHY?

Why RecursiveCharacterTextSplitter instead of CharacterTextSplitter?

vs: Recursive vs Character splitter

**IDEAL
ANSWER**

CharacterTextSplitter splits on one separator — if that separator is rare, chunks can be huge. RecursiveCharacterTextSplitter tries \n\n then \n then . then space in order, always finding a natural break. It never cuts mid-sentence, preserving readable, meaningful chunks.

Score: ____ /5

Notes: _____

Q15**What is chunk_overlap and what happens if you set it to 0?**[\[Parameter Tuning\]](#)**IDEAL ANSWER**

Overlap copies the last N characters from one chunk into the start of the next. Without overlap, a key sentence split across a boundary is half in chunk 3 (no context) and half in chunk 4 (no context). 50-char overlap ensures the LLM sees the full sentence in at least one chunk.

Score: ____/5

Red flag:

Notes: _____

Q16

WHY?

Why use different chunk sizes per format (PDF=500, CSV=800, URL=400)?*vs: format-aware vs uniform chunking***IDEAL ANSWER**

PDFs have structured paragraphs — 500 chars preserves them intact. CSV rows are atomic facts — 800 groups a few rows for related context. URLs are noisy with nav/footer junk — 400 keeps only content. Uniform chunking would hurt retrieval quality across formats.

Score: ____/5

Notes: _____

Q17

WHY?

Why PyPDFLoader instead of pdfplumber or PyMuPDF?*vs: PyPDFLoader vs pdfplumber vs PyMuPDF***IDEAL ANSWER**

PyPDFLoader is native LangChain — outputs Document objects with page metadata automatically, plugs directly into the pipeline. pdfplumber handles tables better but adds complexity. PyMuPDF is faster for large files but needs extra system dependencies. For a demo, simplicity wins.

Score: ____/5

Notes: _____

Q18**What metadata does each chunk carry and why does it matter for the user?**[\[Metadata\]](#)**IDEAL ANSWER**

source_type, filename, page number, chunk_index, chunk_hash, char_count, ingested_at. Page number lets users verify the answer in the original document. chunk_hash enables deduplication. filename is shown in source citations so users know which document the answer came from.

Score: ____/5

Red flag:

Notes: _____

Q19

WHY?

Why WebBaseLoader instead of requests + BeautifulSoup directly?*vs: WebBaseLoader vs raw scraping***IDEAL ANSWER**

WebBaseLoader returns LangChain Document objects with metadata already populated — it slots directly into the splitter pipeline with no wrapping code. Raw BeautifulSoup needs manual Document construction. For JS-heavy sites I'd upgrade to Playwright.

Score: ____/5

Notes: _____

Q20**How does a CSV file become searchable chunks?**[\[CSV Ingestion\]](#)**IDEAL ANSWER**

Each row is converted to a text string: 'column_name: value' for every column, joined by newlines — making it readable English. These strings become Documents and go through the standard splitter and embedding pipeline. A query about a product price matches the row containing that product.

Score: ____/5

Red flag:

Notes: _____

Section 4 — RAG Chain & LLM

Day 5–6 — chain types, prompting, memory

Q21

What are the three steps of RAG?

[Concept]

IDEAL
ANSWER

Retrieve: embed the question and find the k most similar chunks in FAISS. Augment: combine those chunks with the question and chat history into a single prompt. Generate: the LLM reads only that prompt and produces a grounded answer with source references.

Score:
___/5

Red flag:

Notes: _____

Q22

WHY?

Why chain_type='stuff' instead of map_reduce or refine?

vs: stuff vs map_reduce vs refine

IDEAL
ANSWER

Stuff pastes all retrieved chunks directly into one prompt — simplest and fastest. With k=4 chunks of 500 chars each, the total context is ~2000 chars, well within GPT-3.5's 16k token limit. map_reduce is needed for 100+ chunks. refine is more accurate but makes multiple LLM calls, costing more.

Score:
___/5

Notes: _____

Q23

WHY?

Why temperature=0 instead of 0.7?

vs: temperature=0 vs 0.7

IDEAL
ANSWER

temperature=0 makes GPT fully deterministic — it always picks the highest-probability token. For factual document Q&A, we want consistent, reproducible answers, not creative variation. temperature=0.7 would risk the LLM embellishing or contradicting the source document.

Score:
___/5

Notes: _____

Q24

WHY?

Why switch to ConversationalRetrievalChain on Day 6?

vs: ConversationalRetrievalChain vs RetrievalQA

IDEAL
ANSWER

RetrievalQA has no memory — every question is independent. If the user asks 'What is it about?' then 'Tell me more about that,' the second question has no referent for 'that.' ConversationalRetrievalChain reformulates follow-ups using chat history before retrieving, making multi-turn conversation work naturally.

Score:
___/5

Notes: _____

Q25

WHY?

Why window memory of 3 exchanges instead of keeping all history?

vs: window=3 vs unlimited memory

IDEAL
ANSWER

Each stored exchange adds tokens to every subsequent prompt. Unlimited memory causes prompts to grow until they hit the context limit and the API errors. 3 exchanges covers typical conversational depth while keeping prompt size and cost bounded and predictable.

Score:
___/5

Notes: _____

Q26**How does your system prompt prevent hallucination?***[Prompt Engineering]***IDEAL ANSWER**

Three rules in the system prompt: (1) Use ONLY the context below — prevents drawing from training data. (2) If not in context, say 'I don't have that information in the document' — explicit fallback phrase. (3) Cite which document each fact comes from — adds accountability and verifiability.

Score: ____/5

Red flag:

Notes: _____

Section 5 — FastAPI Backend

*Day 7–8 — API design, validation, error handling***Q27**
WHY?**Why FastAPI instead of Flask?***vs: FastAPI vs Flask vs Django***IDEAL ANSWER**

FastAPI is async by default — LLM calls can take 3–5 seconds and async prevents blocking other requests. It auto-generates Swagger docs at /docs with zero code. Pydantic validation is built in — Flask needs marshmallow or manual checks. For an AI API, FastAPI is the industry standard.

Score: ____/5

Notes: _____

Q28
WHY?**Why separate /upload and /upload/url endpoints?***vs: one endpoint vs separate***IDEAL ANSWER**

File upload uses multipart/form-data (binary). URL ingestion uses a JSON body. These are different HTTP content types that can't cleanly share one endpoint. Separate endpoints give clear validation, error messages, and OpenAPI docs for each case.

Score: ____/5

Notes: _____

Q29**What is a Pydantic model and how did you use it?***[Pydantic]***IDEAL ANSWER**

Pydantic validates data types automatically. ChatRequest ensures 'question' is a non-empty string. ChatResponse guarantees the response has 'answer' (string) and 'sources' (list). If the client sends wrong data types, FastAPI returns a 422 error before any code runs.

Score: ____/5

Red flag:

Notes: _____

Q30**What does CORSMiddleware do and why did you need it?***[Security] [CORS]***IDEAL ANSWER**

CORS is a browser security rule that blocks requests from a different origin (domain+port). Streamlit runs on port 8501, FastAPI on 8000 — different origins. Without CORSMiddleware, the browser rejects every Streamlit call to the API before it even reaches FastAPI.

Score: ____/5

Red flag:

Notes: _____

Q31
WHY?

Why a global chat_chain variable instead of creating a new chain per request?

vs: global vs per-request chain

**IDEAL
ANSWER**

Building the chain loads the FAISS index and initialises the LLM — adds 1–2 seconds per message. The global chain initialises once on first request and is reused. Trade-off: not thread-safe for multiple simultaneous users — in production I'd use a per-session chain pool.

Score: ____ /5 Notes: _____

Q32

How would you handle a 500MB PDF upload gracefully?

[Scalability]

**IDEAL
ANSWER**

Add file size validation in the endpoint — return 413 if over limit. Move processing to a BackgroundTask so the HTTP response returns immediately. Return a job_id the client polls via GET /status/{job_id}. For very large indexes, switch from in-memory FAISS to a disk-based solution like FAISS with mmap.

Score: ____ /5 **Red flag:** Notes: _____

Section 6 — Multi-Document Support

Day 10 — FAISS merging, registry

Q33
WHY?

Why merge_from() instead of rebuilding the index on each upload?

vs: merge_from() vs full rebuild

**IDEAL
ANSWER**

Rebuilding re-embeds every previously uploaded document — wasting API calls proportional to total documents. merge_from() merges a new small index into the existing one in milliseconds with no re-embedding. Trade-off: individual documents can't be removed without a full rebuild.

Score: ____ /5 Notes: _____

Q34

What is registry.json and why is it needed?

[Data Management]

**IDEAL
ANSWER**

A JSON file tracking each document's name, source type, chunk count, and ingestion timestamp. Without it there's no way to know what's inside the FAISS index — the index is a black box of vectors. The registry powers the /documents endpoint and the sidebar document list.

Score: ____ /5 **Red flag:** Notes: _____

Q35

If two documents contradict each other, what does the chatbot say?

[Edge Cases]

**IDEAL
ANSWER**

FAISS returns the most similar chunks from both. The LLM receives context from both and the system prompt instructs it to cite each source. A good LLM will note the disagreement: 'doc1.pdf states X while the Wikipedia article states Y.' The user can then judge.

Score: ____ /5 **Red flag:** Notes: _____

Q36

WHY?

Why keep uploaded files on disk instead of deleting after processing?*vs: keep files vs discard***IDEAL
ANSWER**

Files enable re-ingestion without re-upload after a server restart, auditing of what data was ingested, and debugging — if a chunk looks wrong, you open the original. Trade-off: disk usage grows. The /reset endpoint clears both the index and the uploads folder.

Score: ____ /5
Notes: _____

Section 7 — LangChain vs LlamaIndex

*Day 11 — framework comparison***Q37**

WHY?

Why LangChain as default and LlamaIndex as optional?*vs: LangChain vs LlamaIndex as primary***IDEAL
ANSWER**

LangChain has built-in ConversationBufferWindowMemory, full custom prompt control, and clean FastAPI integration. LlamaIndex abstracts too much — injecting a custom system prompt requires fighting the framework. For an API-first chatbot, LangChain's explicit pipeline is the better fit.

Score: ____ /5
Notes: _____

Feature	LangChain	LlamaIndex
Setup complexity	Medium — more code	Low — SimpleDirectoryReader
Custom prompts	Full control	Limited by abstraction
Memory	Built-in	Manual implementation
FastAPI fit	Excellent	Good
Advanced retrieval	Good	Sub-question + recursive
Best for	APIs + pipelines	Document Q&A prototypes

Q38**How did you switch backends at runtime without restarting the API?***[API Design]***IDEAL
ANSWER**

Added ?backend=llamaindex as a query parameter to the /chat endpoint. The endpoint checks this parameter: if 'llamaindex', it loads the LlamaIndex VectorStoreIndex and calls query_engine.query(). Otherwise it uses the default LangChain chain. No restart, no state change needed.

Score: ____ /5
Red flag: Notes: _____

Section 8 — Streamlit Frontend

Day 12 — UI, session state, API integration

Q39

WHY?

Why Streamlit instead of React or Vue?

vs: *Streamlit vs React/Vue*

**IDEAL
ANSWER**

Streamlit lets a Python developer build a working UI in hours without learning JavaScript. For a portfolio project demonstrating RAG skills, the AI pipeline matters more than the UI framework. In production with many users, React would be better for performance and real-time streaming.

Score:
___/5

Notes: _____

Q40

How does chat history persist between user messages in Streamlit?

[Session State]

**IDEAL
ANSWER**

Streamlit reruns the entire Python script on every interaction. `st.session_state` is a persistent dictionary that survives reruns. I store the messages list in `session_state` — each rerun reads it and re-renders the full conversation from top to bottom.

Score:
___/5

Red flag: Notes: _____

Q41

WHY?

Why read `API_URL` from an environment variable?

vs: *env var vs hardcoded URL*

**IDEAL
ANSWER**

In Docker, the frontend container can't reach the backend via 'localhost' — it must use the Docker service name 'backend'. The environment variable lets the same Streamlit code work locally (`localhost:8000`) and in Docker (`backend:8000`) without any code change.

Score:
___/5

Notes: _____

Q42

How do source citations appear in the Streamlit UI?

[UI/UX]

**IDEAL
ANSWER**

After each assistant message, I render a collapsible `st.expander` showing each source chunk with its filename, page number, and first 200 characters of content. This lets the user verify the answer against the original document without leaving the app.

Score:
___/5

Red flag: Notes: _____

Section 9 — Docker & Deployment

Day 13–14 — containers, networking, HF Spaces

Q43

WHY?

Why Docker instead of just sharing `requirements.txt`?

vs: *Docker vs requirements.txt only*

IDEAL ANSWER	requirements.txt only captures Python packages — not the Python version, OS libraries, system dependencies, or runtime environment. Docker freezes the entire runtime. Anyone runs 'docker-compose up --build' and gets the identical environment regardless of OS or existing Python installations.
Score: ____ /5	Notes: _____
Q44 WHY?	Why two containers instead of running FastAPI and Streamlit in one? <i>vs: two containers vs monolith</i>
IDEAL ANSWER	Separation of concerns: if the UI crashes, the API keeps serving. Each service scales independently. If 1000 users hit the frontend and 10 hit the API, you can scale containers separately. It also mirrors real production architecture. Trade-off: requires Docker networking config.
Score: ____ /5	Notes: _____
Q45	Why can't the frontend use 'localhost' to reach the backend in Docker? <i>[Docker Networking]</i>
IDEAL ANSWER	Each container has its own network namespace — localhost inside the frontend container refers to that container only, not the host machine. Docker Compose creates a virtual bridge network where containers reach each other by service name. 'backend:8000' is the correct address.
Score: ____ /5	Red flag: _____ Notes: _____
Q46 WHY?	Why mount faiss_index as a Docker volume? <i>vs: volumes vs bake into image</i>
IDEAL ANSWER	The FAISS index is generated at runtime from user uploads — it doesn't exist at build time and changes with every upload. A volume binds a host directory into the container so data survives container restarts and image rebuilds. Without it, every restart loses all uploaded documents.
Score: ____ /5	Notes: _____
Q47	How did you store the OpenAI API key securely in Docker and on Hugging Face? <i>[Security]</i>
IDEAL ANSWER	Locally: env_file: - .env in docker-compose.yml reads from a .env file that is git-ignored (never committed). On HF Spaces: added OPENAI_API_KEY as a Repository Secret in Settings — HF injects it as an environment variable. The key never appears in code or git history.
Score: ____ /5	Red flag: _____ Notes: _____
Q48 WHY?	Why a separate app.py at root for HF Spaces instead of using frontend/app.py directly? <i>vs: HF Spaces single-process constraint</i>
IDEAL ANSWER	HF Spaces expects a single app.py entry point and provides only one process. Our project needs both FastAPI (port 8000) and Streamlit (port 8501) running simultaneously. app.py starts the FastAPI server in a background thread, waits 4 seconds, then launches Streamlit — two services, one process.
Score: ____ /5	Notes: _____
Q49	What is a Docker health check and how did you configure it? <i>[DevOps] [Reliability]</i>

IDEAL ANSWER	A command Docker runs periodically to verify the container is working. I configured 'curl -f localhost:8000/health' every 30 seconds. After 3 consecutive failures, Docker marks the container unhealthy. The frontend's 'depends_on: condition: service_healthy' ensures Streamlit only starts after a healthy backend.	
Score: ____ /5	Red flag:	Notes: _____

Section 10 — Debugging & Problem Solving

Real-world scenarios the interviewer will probe

Q50	The chatbot gives a wrong answer. Walk me through debugging it step by step. <i>[Debugging]</i>	
IDEAL ANSWER	Step 1: print log_retrieved_chunks() — are the right chunks being fetched? Step 2: If wrong chunks — check chunk size, overlap, embedding model, or k value. Step 3: If right chunks but wrong answer — check system prompt wording and temperature. Step 4: If empty results — verify FAISS index was built and saved correctly.	
Score: ____ /5	Red flag:	Notes: _____

Q51	A user says the answer is on page 5 but the bot says it doesn't know. What happened? <i>[Debugging] [Retrieval]</i>	
IDEAL ANSWER	The key sentence was probably split across a chunk boundary, or the chunk was shorter than min_chars and got filtered out, or k=4 didn't reach the relevant chunk. Fix: increase k, reduce min_chars threshold, or increase chunk_overlap so boundary sentences appear in at least one complete chunk.	
Score: ____ /5	Red flag:	Notes: _____

Q52 WHY?	Why not set k=20 to always retrieve more chunks? <i>vs: k=4 vs k=20</i>	
IDEAL ANSWER	More chunks = longer prompt = higher token cost + slower response. Beyond k=6-8, the LLM suffers from 'lost in the middle' — it ignores chunks in the middle of a long context. The optimal k is the smallest value that reliably retrieves the relevant chunk for your chunk size.	
Score: ____ /5	Notes: _____	

Q53	A user uploads a scanned PDF with no text — what goes wrong and how would you fix it? <i>[Edge Cases] [OCR]</i>	
IDEAL ANSWER	PyPDFLoader returns empty page strings since it extracts text layer only. Fix: detect empty pages after loading, then run OCR on page images using pytesseract or send to AWS Textract. I'd add a check in pdf_loader.py: if len(page.page_content.strip()) == 0, flag for OCR processing.	
Score: ____ /5	Red flag:	Notes: _____

Q54 WHY?	Why not use pgvector or Pinecone instead of FAISS for production? <i>vs: FAISS vs pgvector vs Pinecone (production)</i>	
--------------------	---	--

IDEAL ANSWER

FAISS is perfect for single-user demos — zero infrastructure, just two files. pgvector adds SQL + vector search in one database, enabling filtering by user/date and multi-user isolation, but requires a PostgreSQL server. Pinecone is fully managed but costs money. Migrate when you need multi-user or metadata filtering.

Score: ____ /5

Notes: _____

Q55

How would you add multi-user support so each user's documents are private?
[Scalability] [Architecture]

IDEAL ANSWER

Assign each user a session_id (UUID) at login. Store a separate FAISS index per user at faiss_index/{session_id}/. The /upload and /chat endpoints accept session_id and load the correct index. ConversationBufferWindowMemory is stored in a dict keyed by session_id and cleared on logout.

Score: ____ /5

Red flag:

Notes: _____

Q56

If you had two more weeks, what would you improve first?
[Product Thinking]

IDEAL ANSWER

Streaming responses with FastAPI StreamingResponse and Streamlit st.write_stream — so answers appear token by token instead of after a full wait. Second: user authentication with JWT. These two changes make it feel like a real commercial product rather than a demo.

Score: ____ /5

Red flag:

Notes: _____

SCORING SUMMARY

Section	WHY Qs	Total Qs	Max	Score
1. Project Overview	1	5	25	____ / 25
2. Embeddings & FAISS	3	7	35	____ / 35
3. Ingestion & Chunking	4	8	40	____ / 40
4. RAG Chain & LLM	3	6	30	____ / 30
5. FastAPI Backend	3	6	30	____ / 30
6. Multi-Document	2	4	20	____ / 20
7. LangChain vs LlamaIndex	1	2	10	____ / 10
8. Streamlit Frontend	1	4	20	____ / 20
9. Docker & Deployment	4	7	35	____ / 35
10. Debugging & Problem Solving	1	7	35	____ / 35
TOTAL	23	59	295	____ / 295

Score	Rating	Decision	What it means
236–295 (80%+)	Excellent	Strong hire	Explains WHY decisions confidently, not just what

Score	Rating	Decision	What it means
177–235 (60%+)	Good	Hire	Solid core understanding, minor gaps in trade-offs
118–176 (40%+)	Average	Maybe	Knows steps but can't justify design choices
Below 118	Weak	Pass	Memorised tutorial without understanding the reasoning

Final Notes

Overall communication clarity: _____

Strongest WHY answer: _____

Weakest section: _____

Red flags noticed: _____

Hire decision: _____